Projeto de Programação Aplicada

1º Ten Nicolas Oliveira
Instituto Militar de Engenharia

15 de outubro de 2026

Monitoramento Inteligente de Carga

Contexto

Você, Oficial Engenheiro Militar, acaba de receber sua primeira missão no setor de desenvolvimento de sistemas embarcados de uma unidade de apoio logístico do Exército Brasileiro.

Uma carga sensível está sendo transportada, e há risco de violação, roubo ou falha ambiental durante o deslocamento. Esta carga precisa ser monitorada remotamente com sensores discretos e confiáveis, que permitam detectar qualquer tentativa de violação ou comprometimento de integridade.

Objetivo da Missão

A turma será dividida em grupos de **três integrantes**, e cada grupo ficará responsável por desenvolver uma solução embarcada usando o kit de desenvolvimento STM32MP1 DK1 e sensores específicos. Cada sensor será destinado a monitorar um aspecto crítico da integridade da carga. Ao final, todos os sensores serão integrados em um único sistema de monitoramento centralizado, reforçando a importância de desenvolver classes bem encapsuladas, documentadas e de fácil integração.

O sistema deverá enviar os dados dos sensores para um servidor central, via rede local, utilizando **comunicação UDP** em tempo real. Valores anômalos detectados indicarão que a carga foi violada ou comprometida. Esta arquitetura segue a linha de **Internet das Coisas (IoT)**, permitindo monitoramento remoto e centralizado.

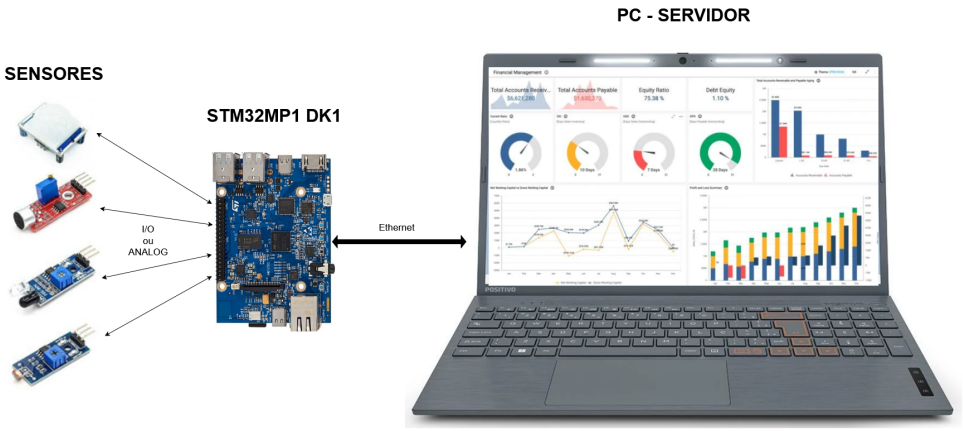


Figura 1 – Digrama do Trabalho

Sensores e Finalidade

Cada grupo receberá um dos sensores abaixo, com a finalidade indicada no contexto de monitoramento remoto de cargas:

1. **Módulo GPS (GY-GPS6MV2):** registrar a localização da carga, permitindo rastreamento e verificação se houve deslocamento não autorizado.
2. **Módulo Sensor Detector de Som - Analógico ou Digital (KY-037):** captar ruídos próximos à carga, possibilitando identificar tentativas de abertura, impacto ou movimentações com ferramentas.
3. **Módulo Sensor de Obstáculo Infravermelho (Reflexão) - Digital (HW-201):** detectar aproximação ou abertura de compartimentos por meio da interrupção/reflexão do feixe IR.
4. **Módulo Sensor de Chuva - Analógico ou Digital:** identificar infiltração de líquidos ou exposição a ambientes úmidos que possam comprometer a integridade da carga.
5. **Módulo Sensor de Força Piezoelétrico - Analógico:** detectar movimentação da carga.
6. **Módulo Sensor de Corrente - Analógico (ACS712):** monitorar consumo elétrico de dispositivos ligados à carga, detectando uso não autorizado ou funcionamento indevido.
7. **Módulo Sensor de Temperatura - Analógico (KY-013):** monitorar condições térmicas; alterações abruptas podem indicar falha de acondicionamento.
8. **Módulo Sensor de Luminosidade - Analógico (LDR KY-018):** detectar abertura do invólucro pela entrada de luz.
9. **Módulo Sensor de Chama - Digital (Flame1100Nm):** identificar a presença de fogo ou calor intenso próximo à carga.
10. **Módulo Sensor de Vibração - Digital (SW-420):** detectar movimentações, impactos ou transporte indevido.
11. **Módulo Sensor de Presença PIR - Digital (AM312):** detectar presença humana próxima à carga, indicando possível tentativa de violação.

Descrição do Projeto

1. Desenvolvimento embarcado

- Compreensão das etapas necessárias para trabalhar com o kit de desenvolvimento e aquisição de dados do sensor.
- Implementação de uma classe C++ para leitura, interpretação e envio dos dados do sensor.
- Compilação cruzada do código para que seja executado em espaço de usuário Linux no kit de desenvolvimento.
- Documentação da código usando Doxygen e encapsulamento da classe para melhor integração no sistema final.

2. Comunicação com o servidor

- A transmissão deve ser contínua e em tempo real.
- Processamento e, se necessário, conversão da leitura bruta para unidades físicas.
- Transmitir os dados via socket UDP, usando um protocolo simples e documentado, para um servidor central.
- O pacote deve conter: identificador do sensor/grupo, valor atual, unidade e, opcionalmente, timestamp.
- O protocolo de comunicação deve ser simples, documentado, e baseado em UDP socket. Sugere-se usar mensagens CSV, JSON, ou binário com campos fixos.

3. Interface gráfica de monitoramento no servidor

Cada grupo deverá criar uma interface gráfica que permita visualizar os dados recebidos no servidor. O objetivo é que, no sistema integrado final, todos os sensores possam ser monitorados de forma centralizada.

A interface poderá ser desenvolvida em **qualquer linguagem de programação ou framework**, desde que atenda aos requisitos funcionais definidos abaixo.

Ferramentas e frameworks recomendados:

- **C++ com Qt**.
- **Python com Tkinter, PyQt ou Matplotlib**.
- **Flutter**.

A escolha da linguagem e framework ficará a critério do grupo, considerando fatores como facilidade de desenvolvimento, portabilidade e capacidade de atender aos requisitos do projeto.

A interface deve conter:

Requisitos obrigatórios

- Valor atual do sensor.
- Histórico gráfico (últimos 30–60 segundos).
- Alerta visual para valores fora de limites pré-definidos.
- Opção de salvar os dados em arquivo de log ou CSV.

Requisitos recomendados (bônus)

- Configuração dinâmica de limites de alerta.
- Exportação automática de relatórios.
- Registro de timestamp.
- Indicador temporal da última atualização recebida.

Instruções Gerais para a Entrega

1. As soluções serão avaliadas considerando:
   - Clareza e simplicidade do código.
   - Conformidade com o enunciado.
   - Uso adequado das técnicas de programação orientada a objeto apresentadas em aula.
2. Não será permitida a utilização de variáveis globais nem a implementação de funcionalidades fora da(s) classe(s) definida(s). O objetivo é avaliar a aplicação correta dos conceitos de programação orientada a objeto.
3. Programas que não compilarem ou que apresentarem erros de execução logo no início serão penalizados de forma significativa. É fundamental que o código seja testado previamente na plataforma de destino.
4. A entrega deverá ser feita através de um repositório Git. O link do **commit final** deve ser informado na entrega via Google Classroom.
5. O arquivo `README.md`, presente no repositório, deverá conter:
   - Descrição geral do projeto.
   - Instruções claras de compilação e execução atualizadas a cada etapa.
   - Estrutura do código e breve explicação das classes e métodos.
   - Dependências necessárias.
6. O `README.md` deve contar, ao fim do projeto: fluxograma, diagrama de classes, instrução de compilação e uso, documentação do projeto e do código, protocolo usado na comunicação (formato das mensagens entre o kit e o computador), descrição do sensor e funcionamento, capturas de tela da interface, e análise dos valores medidos.
7. A documentação e as evidências de todas as entregas devem estar no README.md, incluindo *prints* das interfaces e do terminal com os programas funcionando.

Entregáveis

Cada grupo deve entregar:

Entrega	**Data**	**Valor**
0. Foto da conexão do sensor com o kit.	19/10/2025	
1. Código-fonte em C++ da classe de leitura do sensor, documentada com Doxygen.	26/10/2025	2,0 pontos
2. Programa funcional no kit de desenvolvimento, com leitura dos dados do sensor, com instruções de compilação e execução documentadas no README.md do repositório	02/11/2025	2,0 pontos
3. Inclusão no projeto do envio da leitura do sensor via UDP para o servidor e documentação do protocolo, atualizando as instruções necessárias	09/11/2025	2,0 pontos
4. Implementação da interface gráfica para visualização no PC.	16/11/2025	2,0 pontos
a) Atualização em tempo real da leitura do sensor.		
b) Histórico temporal das últimas leituras.		
c) Salvamento do histórico em arquivos de log.		
5. Vídeo de demonstração e Apresentação do projeto com:	16/11/2025	2,0 pontos
• Fluxograma.		
• Diagrama de Classes.		
• Instruções de compilação e uso.		
• Documentação do projeto, do código e do protocolo.		
• Descrição do sensor e funcionamento.		
• Capturas de tela da interface.		
• Análise dos valores medidos.		

Criação da Máquina Virtual Linux

Para configurar o ambiente de desenvolvimento, siga os passos abaixo:

1. **Criar a máquina virtual:** Utilize o tutorial oficial disponível neste link como referência para criar uma máquina virtual Linux no *VirtualBox*.
2. **Baixar a toolchain:** Acesse o seguinte link do Google Drive e baixe o arquivo arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz. Esta toolchain permitirá a **compilação cruzada** para o kit de desenvolvimento DK1.
3. **Instalar a toolchain na máquina virtual:** Copie o arquivo baixado para a sua máquina virtual e extraia-o com o comando:

```
tar -xvf arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz
```

Utilização do Kit de Desenvolvimento STM32MP1-DK1

A placa **STM32MP157F-DK1** é baseada no processador STM32MP1, amplamente utilizada em aplicações embarcadas. Este kit possibilita o desenvolvimento, teste e validação de aplicações que podem ser portadas para sistemas embarcados.

O acesso à placa pode ser realizado de duas formas principais:

- Via **porta serial** (cabo Micro-USB);
- Via **porta Ethernet** (cabo de rede RJ45).

A alimentação é feita por meio de um cabo USB-C conectado na porta `PWR_IN`. Além disso, a placa dispõe de portas USB tipo A para conexão de periféricos.

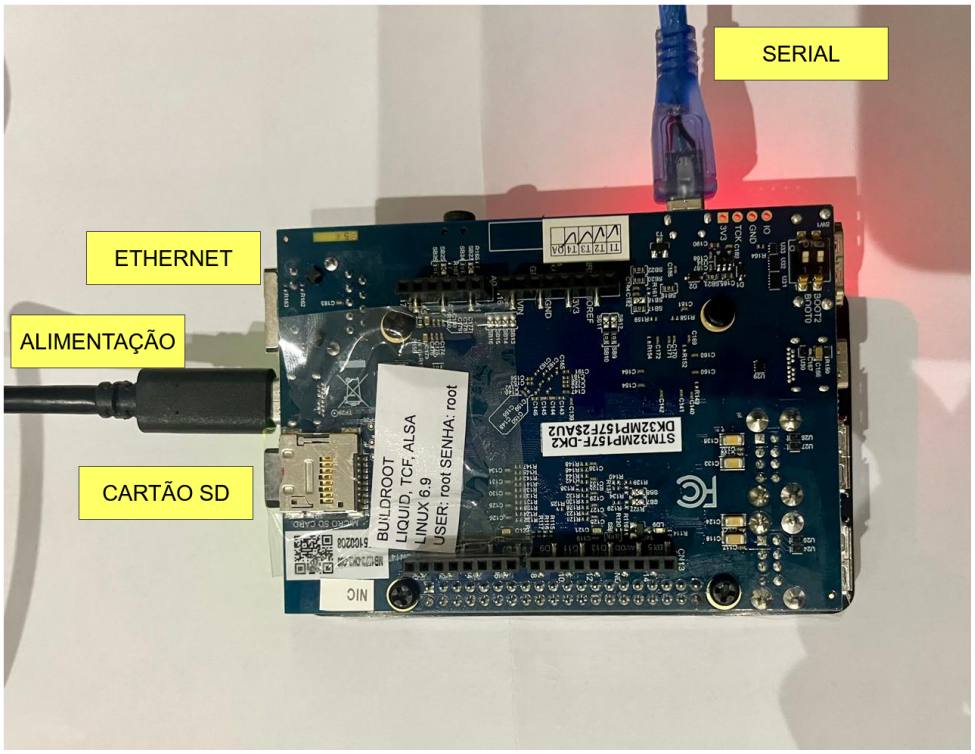


Figura 2 – Diagrama geral da placa STM32MP1-DK1

Alimentação

Conecte um cabo USB-C na porta `PWR_IN` (ao lado da porta Ethernet). Utilize um carregador USB ou a porta USB de um computador para fornecer energia.

Acesso via Serial

1. Conecte um cabo Micro-USB na porta USB Serial do kit.
2. No computador, abra o software `TeraTerm` (ou outro terminal serial).
3. Configure a conexão como **Serial** com os seguintes parâmetros:
    - Baudrate: 115200

- 8N1 (8 bits de dados, sem paridade, 1 stop bit)

4. Pressione `Enter` para acessar o prompt do Linux embarcado.
5. As credenciais padrão são: usuário `root`, senha `root`.

Acesso via Rede (Ethernet)

1. Conecte um cabo Ethernet entre o kit e o roteador.
2. No terminal da placa, execute:

   ```
   ifconfig
   ```

   para verificar o endereço IP (192.168.42.2).
3. Configure no PC um ip na mesma faixa de ip (Ex: 192.168.42.10).
4. No PC, acesse a placa via SSH:

   ```
   ssh root@<ip_da_placa>
   ```

Transferência de Arquivos

Para enviar arquivos do computador para o kit via rede, utilize:

```
scp <arquivo_local> root@<ip_da_placa>:/home/root
```

Portando sua Aplicação

1. Compile o programa em C++ utilizando a **toolchain fornecida** (sysroot + cross-compiler).
2. Copie o executável para o kit via `scp`.
3. Execute e teste a aplicação diretamente no kit.

ATENÇÃO

É obrigatório enviar ao professor uma foto mostrando a conexão do sensor com o kit **desligado**, antes de realizar qualquer teste com o kit energizado.
Uma ligação incorreta pode danificar o kit. Verifique sempre antes de energizar.
Além disso, **os sensores devem sempre ser alimentados com 3,3V**. **Jamais utilize 5V**, pois isso pode danificar permanentemente a placa STM32MP1-DK1. Não retire o cartão SD presente no kit e tome cuidado para não perdê-lo.

Descobrindo o Canal do ADC

A leitura dos sensores é feita pelo subsistema IIO do Linux, disponível no diretório:

```
/sys/bus/iio/devices/iio:device0/
```

Os arquivos `in_voltageX_raw` representam os canais ADC. Para identificar qual canal corresponde ao seu sensor:

1. Conecte o pino do ADC diretamente ao GND.

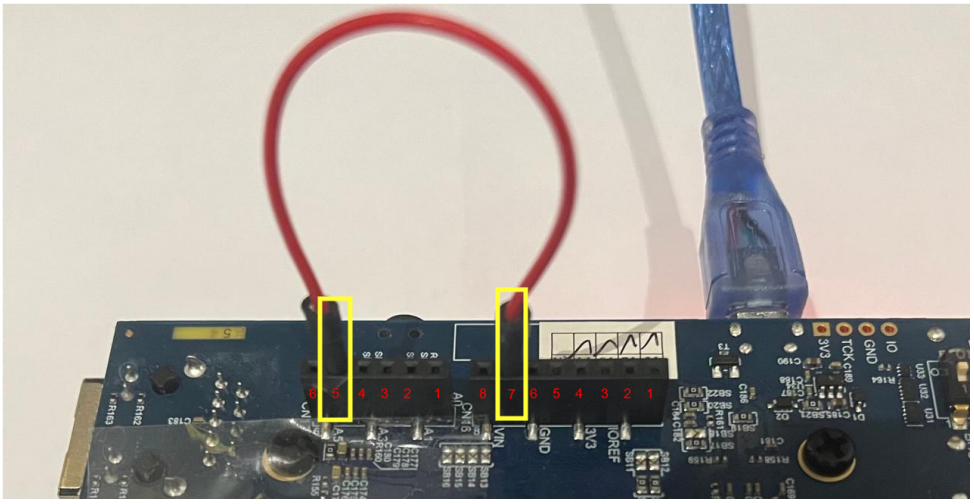


Figura 3 – Conexão de do GND ao ADC do kit

2. Leia os arquivos em `/sys/bus/iio/devices/iio:device0/` com:

   ```
   cat in_voltageX_raw
   ```

3. O arquivo que retornar valor zero corresponde ao canal testado.

Leitura da Serial

Para o caso de sensores que utilizam comunicação serial (como o GPS) leia o arquivo `/dev/ttySTM2` com:

```
cat /dev/ttySTM2
```

Conexão de Sensores

As conexões dos sensores analógicos e/ou digitais devem ser feitas **exatamente** como ilustrado nas imagens abaixo:

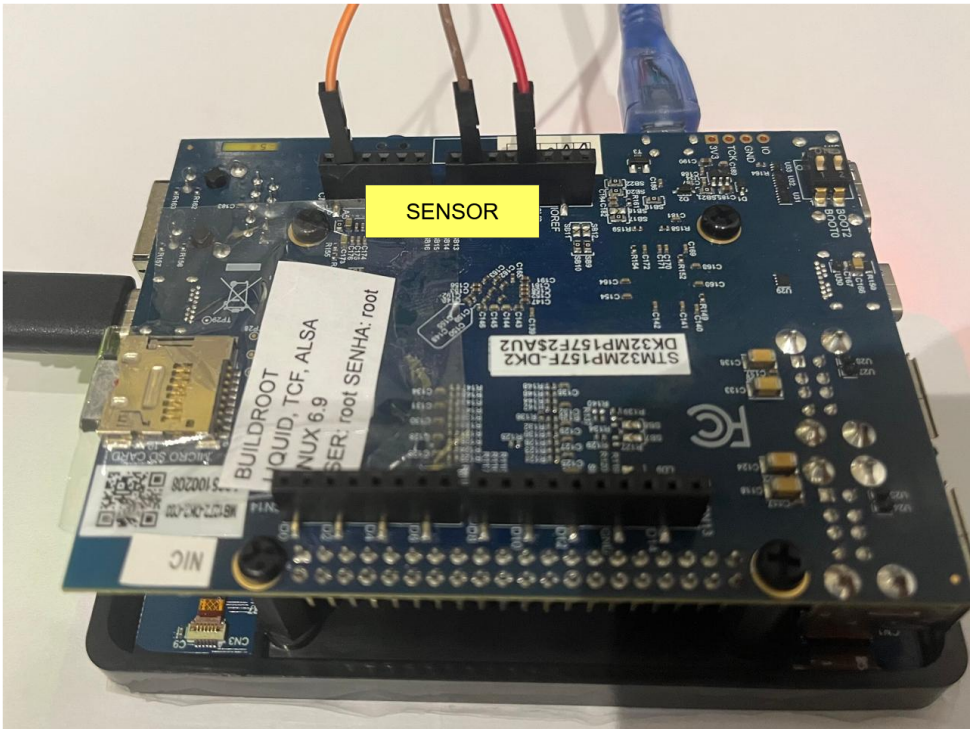


Figura 4 – Conexão de sensores ao ADC do kit

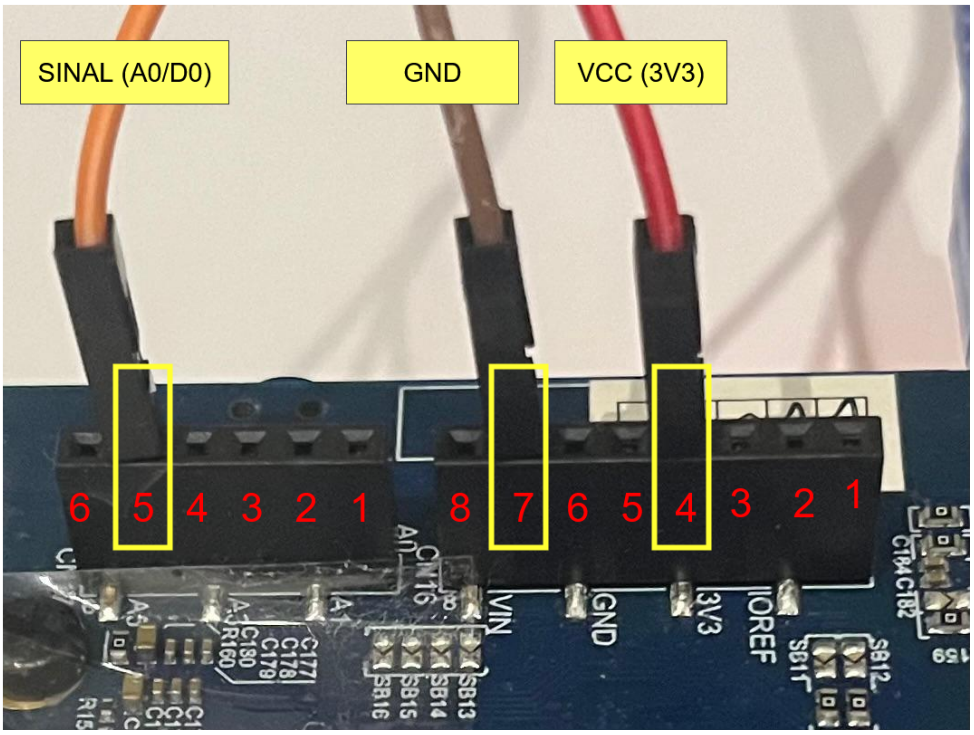


Figura 5 – Conexão de sensores ao ADC do kit

- VCC: 3.3V
- GND: Terra

- Sinal: conectado ao ADC do kit

Já as conexões dos sensores que usam comunicação serial (como o GPS) devem ser feitas **exatamente** como ilustrado nas imagens abaixo:

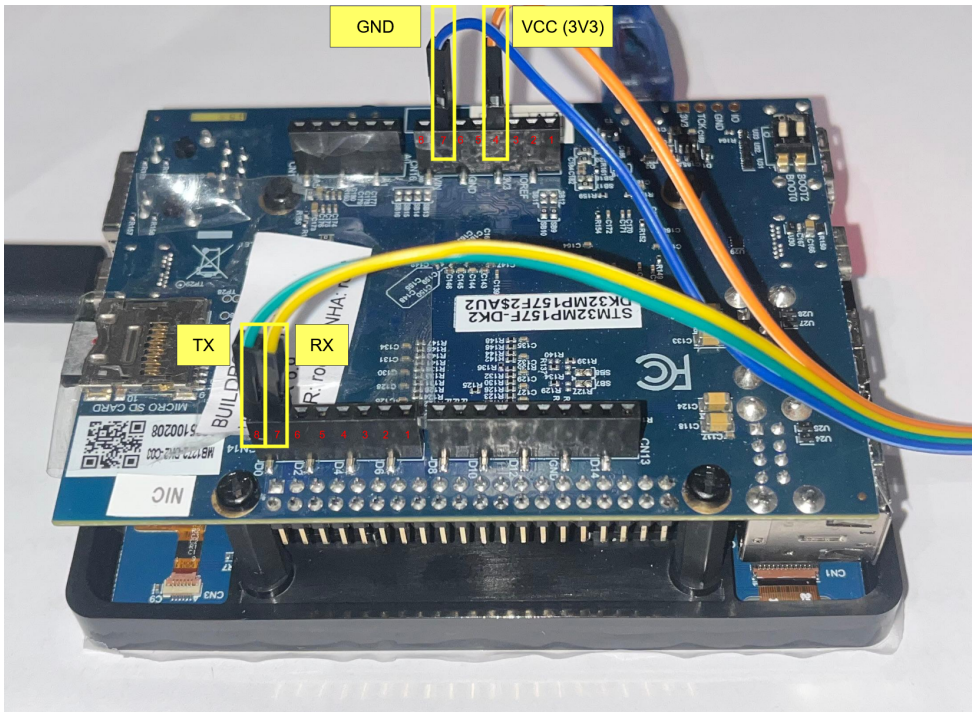


Figura 6 – Conexão de sensores que usam comunicação serial com o kit

- VCC: 3.3V
- GND: Terra
- RX: RX indicado no sensor
- TX: TX indicado no sensor

Resumo

- Alimente com USB-C;
- Acesse via Serial (TeraTerm) ou Rede (SSH);
- Transfira arquivos com `scp`;
- Conecte sensores conforme imagens de referência;
- Descubra o canal do ADC via `in_voltageX_raw`;
- Teste sua aplicação embarcada.

Observações Finais

- Todos os kit de desenvolvimento serão previamente configurados pelo professor.
- O envio dos dados será via rede local (Ethernet) para o servidor central.
- O sistema deve ser modular para permitir integração entre os grupos.
- Ao final, será realizada uma simulação prática com todos os sensores ativos, para avaliar a integridade da carga monitorada.